SIMULATION-BASED EVALUATION OF VERTICAL AND DISTRIBUTED ARCHITECTURES

by

Kaan Yavaş

Emirhan Türkol

Ahmet Can Akay

CSE4297 Engineering Project 1

Project Specification Document

Supervised by:

Doç. Dr. MURAT CAN GANİZ

Marmara University, Faculty of Engineering
Computer Engineering Department

16.04.2026

Copyright © Group members listed above, 2026. All rights reserved.

Contents

1 Problem Statement 1

2 Problem Description and Motivation 1

3 Main Goal and Objectives 2

4 Related Work 4

5 Scope 6

6 Methodology and Technical Approach 7

6.1 System Modeling 8

6.1.1 Vertical Architecture Model (Scale-up) 8

6.1.2 Distributed Architecture Model (Scale-out) 9

6.2 Simulation Method 9

6.3 Input Parameters and Evaluation 9

6.4 Performance Evaluation 10

6.5 Resources 10

7 Professional Considerations 11

7.1 Methodological Considerations / Engineering Standards 11

7.2 Realistic Constraints 11

7.3 Broader Engineering Constraints 12

7.4 Risk Awareness in Design Decisions 12

8 Management Plan 13

8.1 Description of Task Phases 13

8.2 Division of Responsibilities and Duties among Team Members ... 14

8.3 Timeline ... 14

9 Success Factors and Risk Management ... 15

9.1 Measurability/Measuring Success ... 15

9.2 Risk Management ... 16

10 Benefits and Impact of the Project ... 17

References ... 18

1 Problem Statement

This project develops a simulation framework that analyzes the choice between a vertically scaled single-machine system and a horizontally scaled distributed cluster under given workload and service-level constraints. The framework performs parameter sweeps over inputs such as data size, throughput, latency targets, and availability, and identifies all feasible system configurations that satisfy these constraints. Among these, it determines the minimum-cost solutions for each architecture and enables direct comparison through generated metrics and graphs. The core problem is to understand under which input conditions each architectural approach becomes feasible and preferable.

2 Problem Description and Motivation

The design of large-scale computing systems requires selecting an appropriate architectural approach that can satisfy given workload characteristics and service-level objectives. Two fundamental paradigms dominate this space: vertically scaled systems (single, high-capacity machines with shared resources) and horizontally scaled distributed clusters (multiple smaller nodes coordinated over a network). Each approach exhibits distinct performance, cost, and reliability characteristics, and their suitability depends on the interaction between workload properties—such as throughput demand, latency constraints, data size, and access patterns—and system-level requirements including availability and consistency. Understanding this relationship is essential for making informed architectural decisions.

The motivation for this project stems from the fact that architectural choice forms the foundation of an IT system and has long-term implications for performance, scalability, operational complexity, and cost. Once a system is built around a particular architectural paradigm, changing it becomes difficult and expensive. Therefore, such decisions should not be made heuristically or based solely on trends, but rather through careful, systematic analysis. This project aims to address this need by developing a simulation-based framework that models and evaluates these architectural alternatives under controlled and explicitly defined conditions.

The project is worthwhile because it provides a structured and reproducible method for analyzing a critical design decision that is typically handled in an ad hoc manner. By exploring a wide parameter space and identifying feasibility regions for each architecture, the framework enables a deeper understanding of when and why one approach becomes preferable over the other. This contributes not only to practical decision-making but also to a more principled understanding of system behavior under varying workload and service constraints.

In this study, we plan to construct abstract yet representative models of vertical and distributed systems, calibrated to reflect real-world performance trends. Using a discrete-event simulation approach, the framework will accept inputs such as throughput targets, latency constraints (p95/p99), availability requirements, consistency levels, data size, working set characteristics, and traffic burstiness. For each set of inputs, the system will generate candidate configurations, evaluate their performance, and identify all feasible solutions that satisfy the specified constraints. Among these, the minimum-cost configurations for each architectural approach will be selected and compared. Additionally, the framework will support parameter sweep analyses and produce visualizations that reveal how architectural feasibility and optimality shift across different regions of the input space.

3 Main Goal and Objectives

The main goal of this project is to develop a simulation-based framework for analyzing and comparing vertically scaled systems and horizontally scaled distributed clusters under explicitly defined workload and service-level constraints. The framework aims to determine, for a given set of inputs, which architectural approach can provide a feasible solution at lower cost, and to identify the conditions under which the preferred architectural choice changes.

Objective 1: To define the input parameter set of the framework, including throughput targets, latency constraints, availability requirements, consistency level, read/write ratio, data size, working set characteristics, traffic burstiness, replication requirements, and disaster recovery constraints.

Objective 2: To design abstract but representative system models for vertical and distributed architectures, reflecting their key operational characteristics such as centralized resource usage, shared-memory behavior, network communication, replication, and overhead coordination.

Objective 3: To implement a discrete-event simulation framework that models request arrivals, service processes, communication delays, and failure-related behavior over time.

Objective 4: To generate and evaluate candidate system configurations for both architectural approaches under the same workload and SLA definitions.

Objective 5: To determine the feasible configurations that satisfy all specified service-level and system constraints.

Objective 6: To identify the minimum-cost feasible configuration for the vertical architecture and the minimum-cost feasible configuration for the distributed architecture for each simulation scenario.

Objective 7: To compare the two architectural approaches in terms of feasibility, cost, performance, and system behavior under identical input conditions.

Objective 8: To perform parameter sweep analyses over selected input dimensions in order to observe how feasibility and architectural preference change across different workload and SLA conditions.

Objective 9: To produce graphs and other visual outputs that illustrate configuration feasibility, cost trends, and transition regions between the two architectural approaches.

Objective 10: To provide a systematic and reproducible basis for architectural evaluation through clearly defined models, parameters, and simulation procedures.

4 Related Work

The problem of choosing between vertically scaled systems (scale-up) and distributed clusters (scale-out) has been studied from multiple perspectives in the literature. However, these perspectives have largely evolved independently and address only partial aspects of the broader architectural decision problem.

Early work in computer architecture focuses on the theoretical limits of parallelism. Amdahl's Law establishes that the speedup of any parallel system is fundamentally constrained by the serial portion of the workload, implying diminishing returns for both multi-core systems and distributed clusters. Extensions of this theory to multicore and NUMA systems further highlight bottlenecks such as memory bandwidth and cache coherence. While these studies provide important theoretical bounds, they do not address how different architectures behave under realistic workload and system constraints.

In database systems, the focus shifts to architectural design. The classification of shared-memory, shared-disk, and shared-nothing systems provides a structural framework for comparing centralized and distributed approaches. Shared-nothing architectures, as advocated by Stonebraker, aim to minimize contention through partitioning, while scale-up systems rely on tightly coupled resources. DeWitt and Gray further formalize these concepts through scale-up and scale-out efficiency. However, these works remain largely conceptual and do not provide a quantitative mechanism for determining architectural suitability under varying workloads.

Empirical studies in OLTP systems provide more concrete insights into these trade-offs. For example, "OLTP Through the Looking Glass" shows that coordination overheads often dominate execution time, while "OLTP on Hardware Islands" demonstrates that the relative performance of shared-everything and shared-nothing systems depends heavily on workload characteristics. Distributed systems perform well for highly partitionable workloads, whereas centralized systems can be more efficient under skewed or communication-intensive workloads. Despite these

insights, such studies are limited to specific experimental setups and do not generalize into a systematic evaluation framework.

Research in distributed systems further explores the internal behavior of large-scale clusters. Consensus protocols such as Paxos highlight the inherent cost of coordination, while systems like Spanner and Aurora provide practical evidence of how replication and communication affect latency and throughput. Although these works offer detailed insights into distributed system performance, they primarily focus on improving distributed architectures rather than comparing them with scale-up alternatives under equivalent constraints.

Benchmarking studies represent another important line of work. Standard benchmarks such as TPC-C, TPC-E, and TPC-DS provide widely accepted performance metrics, while SPEC and YCSB evaluate processor efficiency and latency-throughput behavior. Although these benchmarks enable reproducible comparisons, they are inherently static and evaluate fixed configurations. As a result, they do not provide a systematic way to analyze how architectural performance changes across varying workloads and service-level requirements.

Vendor documentation and industrial systems offer additional practical insights. Mainframe systems such as IBM Z emphasize high throughput and reliability within a single system, while distributed cloud systems such as Spanner, Aurora, and Azure Hyperscale focus on scalability and fault tolerance. Hybrid systems further blur the distinction between scale-up and scale-out architectures. However, vendor materials are typically not designed for objective comparison and must be interpreted cautiously.

Despite the extensive literature, a clear gap remains. Existing studies focus either on theoretical limits, specific architectures, or fixed experimental evaluations, but very few integrate these perspectives into a unified framework. In particular, there is a lack of approaches that systematically analyze architectural trade-offs across varying workload characteristics and service-level constraints.

This project addresses this gap by proposing a simulation-based framework that models both vertical and distributed architectures in a unified and parameterized manner. Unlike prior work, which evaluates fixed systems, the proposed approach

explores a continuous parameter space and evaluates architectures under identical input conditions. Furthermore, the evaluation is formulated as a feasibility problem, where configurations must satisfy constraints on throughput, latency, availability, and consistency.

Another key contribution is the use of parameter sweep analysis to identify transition points between architectures. Instead of producing isolated comparisons, the framework reveals regions where one architecture becomes more advantageous than the other, enabling systematic analysis of trade-offs.

In summary, the novelty of this project lies in integrating abstract system modeling, discrete-event simulation, feasibility-based evaluation, and parameter sweep analysis into a unified framework for comparing scale-up and scale-out architectures under varying conditions.

5 Scope

The project does not include the implementation of real-world distributed systems, deployment on physical infrastructure, or integration with production environments. It also does not aim to replicate vendor-specific systems or provide exact performance predictions for specific hardware. Instead, it operates on calibrated abstractions that approximate general system behavior. Low-level implementation details such as operating system scheduling, exact network protocols, or hardware-specific optimizations are outside the scope. The framework is not intended to function as a benchmarking tool, but rather as an analytical model for understanding system-level trade-offs.

This project is not directly based on prior implementation or existing software system. However, it is conceptually grounded in established academic work on computer architecture, distributed systems, transaction processing, and performance modeling. External data sources such as vendor documentation and benchmark reports may be used for calibration purposes. In cases where certain parameters—such as detailed cost structures or proprietary system characteristics—are not publicly available, these values will be estimated based on available information and reasonable modeling assumptions.

Several constraints define the boundaries of the project. The simulation is implemented in Python and is expected to run on a standard personal computer, which limits computational complexity and simulation scale. The system relies on abstract cost models rather than real-time pricing data, and performance models are simplified representations of real systems. The input parameter set is fixed to a predefined set of workload and SLA characteristics, and the architectural models are limited to a small number of configurable system classes and node types. Additionally, the accuracy of the results is constrained by the quality of model calibration and the level of abstraction used in the simulation.

The project also involves several assumptions. It is assumed that the defined input parameters (such as workload characteristics and SLA requirements) are sufficient to capture the essential behavior of real-world systems. The simulation assumes that system components behave according to the modeled relationships, including resource scaling, coordination overhead, and failure characteristics. It is also assumed that calibration data derived from external sources reasonably reflects real system trends, even though exact equivalence cannot be guaranteed. Furthermore, the framework assumes that workload characteristics remain stable during each simulation run and that external factors not explicitly modeled (such as unexpected infrastructure anomalies) do not significantly alter system behavior.

6 Methodology and Technical Approach

The project adopts a simulation-based experimental methodology to analyze and compare vertically scaled (scale-up) systems and horizontally scaled distributed (scale-out) systems under identical workload and service-level constraints. Both architectures are treated as alternative solutions to the same formally defined service requirements and are evaluated within a controlled and parameterized environment.

The overall approach consists of four main stages: (i) defining workload and SLA parameters, (ii) constructing abstract system models, (iii) simulating system behavior using a discrete-event simulation (DES) framework, and (iv) evaluating feasible configurations and comparing minimum-cost solutions.

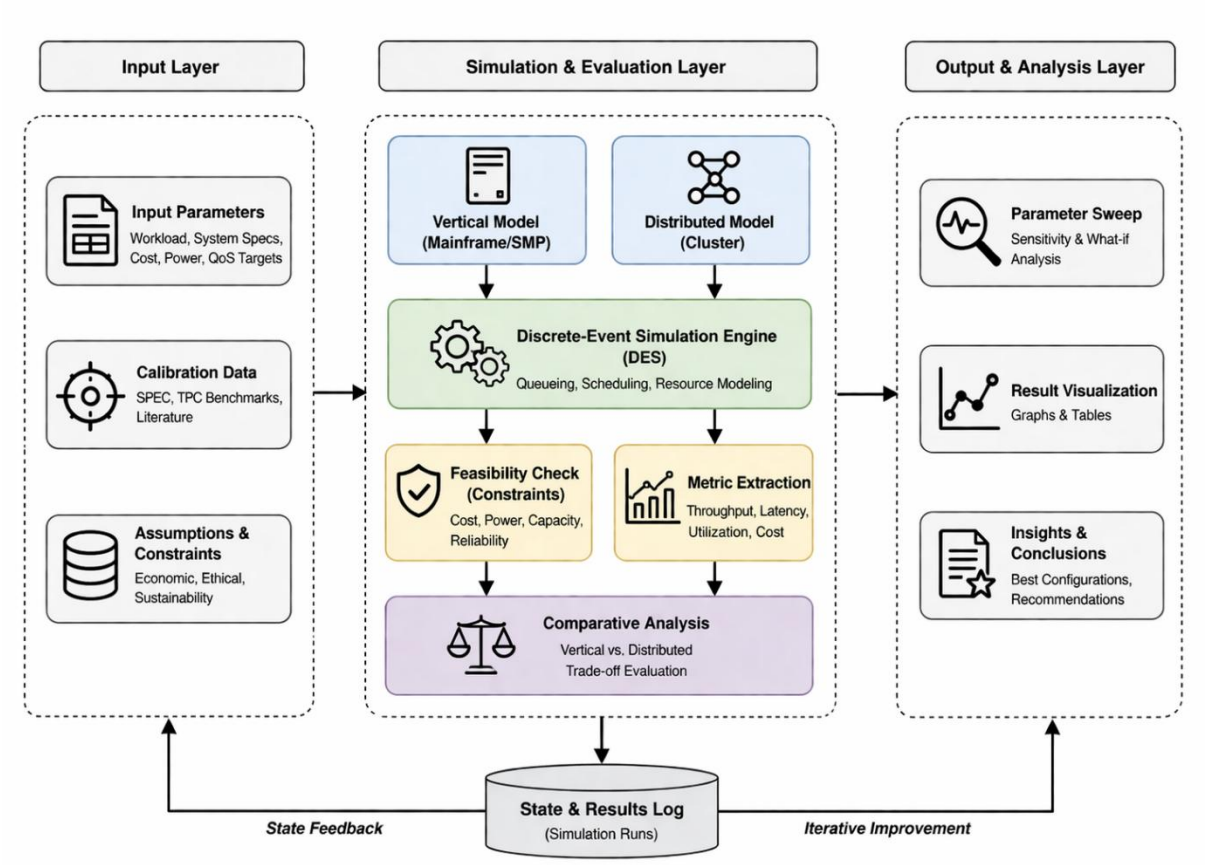


Figure 1: Overall methodology of the proposed simulation framework. (Generated by ChatGPT)

6.1 System Modeling

Two abstract system models are defined:

6.1.1 Vertical Architecture Model (Scale-up)

This model represents a single high-capacity system with shared memory, centralized resource management, and low internal communication latency. Performance is primarily governed by CPU capacity, memory size, and I/O throughput. Degradation mechanisms include memory pressure and data-size-driven access inefficiencies.

6.1.2 Distributed Architecture Model (Scale-out)

This model represents a cluster of independent nodes connected over a network. It explicitly models data partitioning, replication, coordination overhead, and failure handling. System performance scales sublinearly with node count and is influenced by replication requirements, coordination costs, and network delays.

Both models are calibrated using trends from literature and benchmark reports rather than exact vendor-specific measurements.

6.2 Simulation Method

A discrete-event simulation (DES) approach is used to model time-dependent system behavior, including request arrivals, processing, queueing, I/O operations, and network communication. For distributed systems, coordination, replication, and failure-recovery processes are also included.

DES is chosen because it naturally captures time-dependent system behavior, queueing effects, and latency distributions, which are critical for evaluating p99 latency and throughput under bursty workloads.

6.3 Input Parameters and Evaluation

The simulation operates on a fixed set of input parameters representing workload and service-level requirements. These include:

- throughput targets
- latency constraints (p95/p99)
- availability requirements
- consistency level
- read/write ratio
- data size
- working set size

- traffic burstiness
- replication requirements

For each input scenario, multiple system configurations (vertical capacities and distributed cluster setups) are generated and evaluated in terms of throughput, latency, availability, cost, and resource utilization.

A feasibility check is applied based on SLA constraints, and among feasible configurations, the **minimum-cost solution** is selected for each architecture.

6.4 Performance Evaluation

Performance evaluation focuses on comparing architectures under identical conditions, identifying feasibility regions, and analyzing cost-performance trade-offs.

Parameter sweep experiments are conducted by varying key inputs (e.g., data size or workload intensity) to identify threshold effects, sensitivity relationships, and architectural transition points.

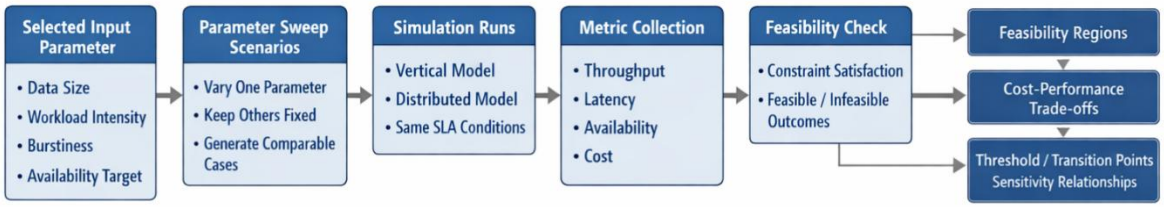


Figure 2: Parameter sweep and performance evaluation process. (Generated by ChatGPT)

6.5 Resources

Software: Python, numerical libraries, plotting tools

Hardware: Standard personal computer

Data Sources: Academic literature, benchmark reports (e.g., TPC-C, TPC-E), vendor documentation

7 Professional Considerations

7.1 Methodological Considerations / Engineering Standards

Modeling Standards: Abstract system modeling based on principles from Computer Architecture and Distributed Systems

Simulation Methodology: Discrete-event simulation and queueing-based modeling are used to represent system dynamics and latency behavior.

Experimental Design: Controlled parameter variation (one-variable-at-a-time and sweep analysis) ensures reproducibility and interpretability.

Version Control: Source code and experiments are managed using Git for traceability and reproducibility.

Documentation and Visualization: Results are presented using structured plots and tables, ensuring clarity in analysis.

Project Planning: Gantt charts and milestone tracking are used for timeline management.

7.2 Realistic Constraints

Several realistic constraints affect the design and outcomes of the project:

Computational Constraints: The simulation must run on a standard personal computer, limiting simulation scale and complexity.

Modeling Limitations: The system models are abstractions and cannot capture all low-level details (e.g., OS scheduling, exact hardware behavior).

Data Availability: Some parameters, especially cost and proprietary system characteristics, are not publicly available and must be estimated.

Calibration Uncertainty: The accuracy of results depends on how well the models reflect real-world trends.

Parameter Scope: The input space is limited to a predefined set of workload and SLA parameters.

7.3 Broader Engineering Constraints

The project also considers constraints relevant to modern engineering practice:

Economic Constraints: Cost modeling is simplified and based on abstract representations rather than real pricing.

Environmental Constraints: Energy consumption is approximated, not physically modeled.

Technological Constraints: The system does not include real deployment or hardware-specific optimizations.

Societal Constraints: The project focuses on analytical insights rather than direct societal applications.

7.4 Risk Awareness in Design Decisions

The design anticipates potential issues:

oversimplified models → mitigated by calibration and sensitivity analysis

missing real-world data → mitigated by estimation and trend alignment

unclear results → mitigated by parameter isolation and sweep experiments

Engineering decisions are made to prioritize **interpretability, reproducibility, and analytical clarity** over unnecessary model complexity.

8 Management Plan

8.1 Description of Task Phases

The project timeline is divided into multiple phases, each including specific tasks, objectives, and deliverables. The phases and their corresponding outputs are described below.

Phase 1: Problem definition, scope determination, identification of input parameters, evaluation metrics, and feasibility criteria.

Deliverables: Defined problem scope, input parameters, SLA constraints, and feasibility criteria.

Phase 2: System design and modeling, including vertical architecture and distributed cluster abstractions, along with modeling assumptions.

Deliverables: System models, architecture diagrams, and modeling assumptions.

Phase 3: Development of initial simulation framework, including base structure, input handling, configuration system, and vertical model implementation.

Deliverables: Initial simulation framework and working vertical model.

Phase 4: Implementation and integration of distributed system model, including scaling, replication, coordination overhead, and unified simulation interface.

Deliverables: Integrated simulation system (vertical + distributed models).

Phase 5: Implementation of simulation logic, including discrete-event simulation, feasibility checking, and optimization mechanisms.

Deliverables: Operational simulation pipeline with feasibility and optimization logic.

Phase 6: Experimentation, analysis, and final documentation, including parameter sweep, performance evaluation, and reporting.

Deliverables: Experimental results, analysis outputs, and final report.

8.2 Division of Responsibilities and Duties among Team Members

This section summarizes how the project tasks and responsibilities are distributed among team members across the main phases of the study.

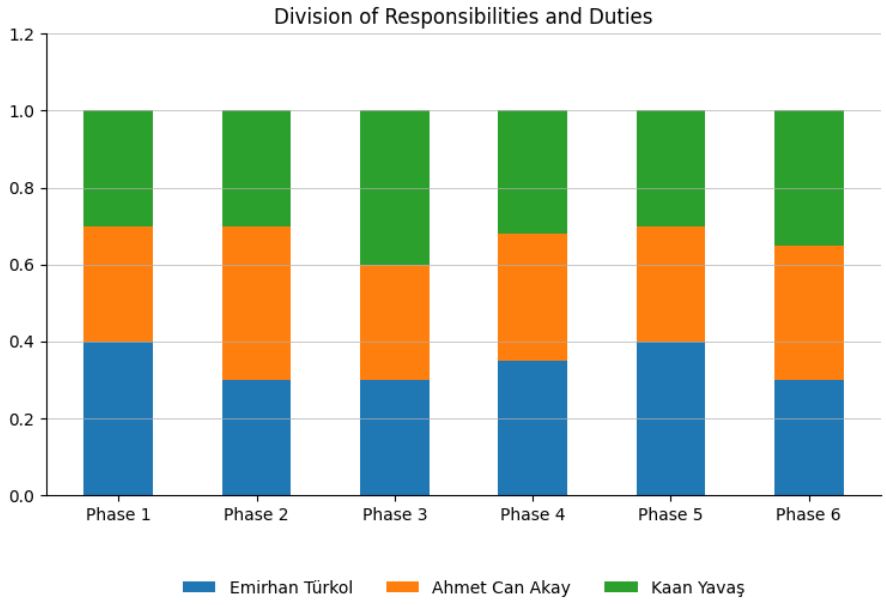


Figure 3: Division of Responsibilities and Duties

8.3 Timeline

This timeline presents the planned sequence and estimated duration of the project phases from problem definition to experimentation and final reporting.

	Feb, 2026	Mar, 2026	Apr, 2026	May, 2026	Jun, 2026	Oct, 2026	Nov, 2026	Dec, 2026	Jan, 2027
Phase 1									
Phase 2									
Phase 3									
Phase 4									
Phase 5									
Phase 6									

Figure 4: Project Timeline

9 Success Factors and Risk Management

9.1 Measurability/Measuring Success

Objective 1: To define and implement a comprehensive input parameter set for the simulation framework, including throughput, latency, availability, consistency, data size, workload characteristics, and replication requirements.

Success Factor 1: All input parameters must be clearly defined, configurable, and actively used within the simulation. Changes in parameter values should result in observable and consistent variations in system outputs.

Objective 2: To design and implement representative system models for both vertical and distributed architectures.

Success Factor 2: The models must produce meaningful outputs (throughput, latency, availability, cost) that respond logically to input changes. The differences between vertical and distributed architectures (e.g., centralization vs. coordination overhead) must be clearly observable in the results.

Objective 3: To develop a discrete-event simulation framework that accurately models system behavior over time.

Success Factor 3: The simulation must successfully execute event-driven workloads and produce consistent results across repeated runs with identical inputs. The system should operate end-to-end without failure.

Objective 4: To generate, evaluate, and filter candidate system configurations based on defined constraints.

Success Factor 4: The framework must generate multiple configurations and correctly evaluate them against constraints such as throughput, latency (p99), and availability. Feasible and infeasible configurations must be clearly distinguishable.

Objective 5: To identify optimal configurations and compare architectural approaches under identical conditions.

Success Factor 5: The system must consistently identify the minimum-cost feasible configuration for each scenario. Comparative results between vertical and distributed architectures must be clear, interpretable, and reproducible.

Objective 6: To perform parameter analysis and produce interpretable outputs and visualizations.

Success Factor 6: The framework must support parameter sweep experiments and generate meaningful graphs that reflect system behavior and transitions. Results must be reproducible and consistent across runs.

9.2 Risk Management

The project may encounter several risks throughout the development and evaluation phases.

Risk 1: Lack of access to reliable real-world calibration data.

Work Package: Model calibration and validation.

Mitigation (Plan B): If real-world data cannot be obtained, the system will rely on publicly available benchmark studies, academic literature, and vendor documentation to approximate system behavior. In the absence of precise values, relative scaling and trend-based estimation will be used to ensure consistency and validity of the simulation outputs.

Risk 2: Oversimplified system models may fail to reflect realistic system behavior.

Work Package: System modeling and simulation design.

Mitigation (Plan B): The models will be iteratively refined by incorporating additional factors such as coordination overhead, memory pressure, and replication effects. Instead of exact numerical validation, model behavior will be compared against known patterns and expectations from existing literature.

Risk 3: No clear or meaningful performance differences observed between architectural approaches.

Work Package: Experiment design and analysis.

Mitigation (Plan B): The input parameter space will be expanded and adjusted (e.g., increasing data size, workload burstiness, or consistency constraints) to expose differences between architectures. Additional scenarios will be introduced to identify transition points and performance trade-offs.

Risk 4: Implementation delays or incomplete system development within the project timeline.

Work Package: Overall system development and integration.

Mitigation (Plan B): A minimal viable framework will be prioritized to ensure that core functionalities (simulation execution, feasibility analysis, and architectural comparison) are completed. Additional features and optimizations will be implemented incrementally if time permits.

10 Benefits and Impact of the Project

The main benefit of this project is to provide a systematic and reproducible framework for evaluating architectural decisions in computing systems. Rather than relying on intuition or ad hoc comparisons, the framework enables analysis based on clearly defined workload characteristics and service-level requirements. In this way, it helps designers and researchers better understand the cost, performance, and feasibility trade-offs between vertical and distributed architectures.

(i) Scientific Impact: The project offers a structured methodology for comparing architectural paradigms under controlled conditions. Its simulation-based approach and focus on feasibility and transition regions may also support future academic studies.

(ii) Economic / Commercial / Social Impact: Although the project is not a direct commercial product, it may help reduce infrastructure costs by supporting better

architectural decisions. It can also serve as an educational and research tool for understanding system design trade-offs.

(iii) Potential Impact on New Projects: The framework can form a basis for future work by being extended with more detailed models, calibration data, or additional architectural alternatives.

(iv) Impact on National Security: The project has no direct national security focus, but it may provide indirect value through improved understanding of system reliability and resilience in critical systems.

References

[1] Standard Performance Evaluation Corporation, *"SPEC CPU 2017,"* 2017.

[2] Standard Performance Evaluation Corporation, *"SPEC Cloud IaaS 2016 Documentation,"* 2016.

[3] Standard Performance Evaluation Corporation, *"SPECjbb2015 User's Guide,"* 2015.

[4] Transaction Processing Performance Council, *"TPC Benchmark C (TPC-C),"* 2026.

[5] Transaction Processing Performance Council, *"TPC Benchmark DS (TPC-DS),"* 2026.

[6] Transaction Processing Performance Council, *"TPC Benchmark H (TPC-H),"* 2026.

[7] B. F. Cooper et al., *"Benchmarking cloud serving systems with YCSB,"* in *Proc. ACM Symposium on Cloud Computing (SoCC)*, 2010, pp. 143–154.

[8] Amazon Web Services, *"Amazon Aurora under the hood: Quorum and correlated failure,"* 2017.

[9] Google Cloud, *"Spanner replication documentation,"* 2026.

[10] Microsoft Corporation, *"Azure SQL Database Hyperscale architecture,"* 2024.

[11] IBM Corporation, *“IBM z16 Technical Introduction,”* Redbooks, 2022.

[12] Hewlett Packard Enterprise, *“HPE Superdome Flex Server architecture and RAS technical white paper,”* 2020.

[13] Oracle Corporation, *“Oracle Exadata X10M Datasheet,”* 2023.

[14] S. Erdem, *“NoSQL veri tabanı sistemlerinin performans analizi ve bir uygulama,”* Yüksek Lisans Tezi, İstanbul Üniversitesi, 2019.

[15] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[16] M. Stonebraker, “The case for shared nothing,” *IEEE Database Engineering Bulletin*, vol. 9, no. 1, pp. 4–9, 1986.

[17] D. J. DeWitt and J. Gray, “Parallel database systems: The future of high performance database systems,” *Communications of the ACM*, vol. 35, no. 6, pp. 85–98, 1992.

[18] J. Gray et al., “The dangers of replication and a solution,” in *Proc. ACM SIGMOD International Conference on Management of Data*, 1996.

[19] L. Lamport, “Paxos made simple,” *ACM SIGACT News*, vol. 32, no. 4, pp. 18–25, 2001.

[20] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[21] A. Pavlo et al., “OLTP through the looking glass, and what we found there,” in *Proc. ACM SIGMOD International Conference on Management of Data*, 2008, pp. 981–992.

[22] I. Porobic et al., “OLTP on hardware islands,” *Proceedings of the VLDB Endowment (PVLDB)*, vol. 5, no. 11, pp. 1447–1458, 2012.

[23] J. C. Corbett et al., “Spanner: Google’s globally-distributed database,” in *Proc. USENIX OSDI*, 2012, pp. 251–264.

[24] A. Verbitski et al., “Amazon Aurora: Design considerations for high throughput cloud-native relational databases,” in *Proc. ACM SIGMOD International Conference on Management of Data*, 2017, pp. 1041–1052.